PYTHON CHEAT SHEET · 02 OF 05

Dictionary (dict)

Key-value mappings with fast lookups: creating, reading, updating, iterating, common patterns, and the Customer Profile Management program.

ORDERING	KEYS	VALUES	LOOKUP
Insertion-ordered (3.7+)	Unique and hashable	Any object, repeats OK	O(1) on average

01 Creating dictionaries

```
>>> person = {"name": "Asha", "age": 28}              # literal
>>> dict(name="Asha", age=28)                         # keyword form (string keys only)
{'name': 'Asha', 'age': 28}
>>> dict([("a", 1), ("b", 2)])                        # from (key, value) pairs
{'a': 1, 'b': 2}
>>> dict(zip(["x", "y"], [10, 20]))                   # from two parallel sequences
{'x': 10, 'y': 20}
>>> dict.fromkeys(["a", "b", "c"], 0)                 # one starting value for every key
{'a': 0, 'b': 0, 'c': 0}
>>> {n: n ** 2 for n in range(1, 5)}                  # dict comprehension
{1: 1, 2: 4, 3: 9, 4: 16}
>>> {}                                                # empty dict
{}
```

TIP Keys must be **hashable**: `str`, `int`, `float`, `bool`, `tuple` (of hashables) and `frozenset` work; `list`, `dict` and `set` do not. `1`, `1.0` and `True` count as the same key.

```
>>> {[1, 2]: "x"}
TypeError: unhashable type: 'list'
>>> {1: "int", 1.0: "float", True: "bool"}            # one key, last value wins
{1: 'bool'}
```

02 Reading values

Task	Syntax	Behaviour
Get by key	`d[k]`	Raises `KeyError` if k is missing
Get safely	`d.get(k)` · `d.get(k, default)`	Returns `None` (or the default) when missing
Test for a key	`k in d` · `k not in d`	Checks keys only, never values
Number of pairs	`len(d)`	
Get or insert	`d.setdefault(k, default)`	Inserts k with the default if missing, returns the value

```
>>> d = {"a": 1, "b": 2, "c": 3}
>>> d["a"]
1
>>> d["z"]
KeyError: 'z'
>>> d.get("z")                                  # None, no exception
>>> d.get("z", 0)                               # custom fallback
0
>>> "b" in d
True
>>> 2 in d                                      # `in` tests KEYS, not values
False
>>> 2 in d.values()
True
>>> len(d)
3
>>> d.setdefault("z", 26)                       # missing: inserted and returned
26
>>> d
{'a': 1, 'b': 2, 'c': 3, 'z': 26}
```

03 Adding, updating and removing

Task	Syntax	Notes
Add or overwrite	`d[k] = v`	Same line works for new and existing keys
Merge in place	`d.update(other)` · `d.update(k=v)`	Values from `other` win on clashes
Merge into a new dict	`d | other`	Python 3.9+; original dicts stay untouched
Merge in place (operator)	`d |= other`	Python 3.9+; like `update`
Delete a key	`del d[k]`	Raises `KeyError` if missing
Remove and return a value	`d.pop(k)` · `d.pop(k, default)`	The default avoids `KeyError`
Remove the last pair	`d.popitem()`	Returns `(key, value)`, last-in first-out (3.7+)
Empty the dict	`d.clear()`	
Shallow copy	`d.copy()` · `dict(d)`	See "Copying" below

```
>>> stock = {"apple": 5, "pear": 2}
>>> stock["kiwi"] = 7                           # add
>>> stock["apple"] += 1                         # update an existing value
>>> stock.update({"pear": 10, "fig": 1})
>>> stock | {"plum": 3}                         # new dict; stock is unchanged
{'apple': 6, 'pear': 10, 'kiwi': 7, 'fig': 1, 'plum': 3}
>>> del stock["fig"]
>>> stock.pop("kiwi")
7
>>> stock.pop("nope", None)                     # default: no KeyError
>>> stock.popitem()                             # the most recently inserted pair
('pear', 10)
>>> stock
{'apple': 6}
```

04 Looping and views

`d.keys()`, `d.values()` and `d.items()` return live **view objects**: they always reflect the dictionary's current contents. Looping over a dict directly gives its keys.

```
>>> d = {"a": 1, "b": 2, "c": 3}
>>> d.keys()
dict_keys(['a', 'b', 'c'])
>>> d.values()
dict_values([1, 2, 3])
>>> d.items()
dict_items([('a', 1), ('b', 2), ('c', 3)])
>>> for key, value in d.items():
...     print(key, "->", value)
a -> 1
b -> 2
c -> 3
>>> list(d)                                          # keys, in insertion order
['a', 'b', 'c']
>>> sum(d.values())
6
>>> sorted(d.items(), key=lambda kv: kv[1], reverse=True)   # sort by value
[('c', 3), ('b', 2), ('a', 1)]
>>> max(d, key=d.get)                                # key that holds the largest value
'c'
>>> {v: k for k, v in d.items()}                     # invert (needs unique, hashable values)
{1: 'a', 2: 'b', 3: 'c'}
>>> {k: v for k, v in d.items() if v > 1}            # filter
{'b': 2, 'c': 3}
```

05 Common patterns

```
>>> words = ["to", "be", "or", "not", "to", "be"]
>>> freq = {}
>>> for w in words:
...     freq[w] = freq.get(w, 0) + 1                 # count occurrences
>>> freq
{'to': 2, 'be': 2, 'or': 1, 'not': 1}
>>> groups = {}
>>> for w in ["apple", "avocado", "banana", "blueberry", "cherry"]:
...     groups.setdefault(w[0], []).append(w)        # group by first letter
>>> groups
{'a': ['apple', 'avocado'], 'b': ['banana', 'blueberry'], 'c': ['cherry']}
>>> user = {"id": 7, "address": {"city": "Pune", "geo": {"lat": 18.5}}}
>>> user["address"]["geo"]["lat"]                    # nested access
18.5
>>> user.get("profile", {}).get("bio", "n/a")        # safe nested access
'n/a'
```

`collections.Counter` and `defaultdict` do the counting and grouping jobs more cleanly: see cheat sheet 03.

06 Copying and JSON

```
>>> import copy
>>> orig = {"tags": ["a"], "n": 1}
>>> shallow = orig.copy()                          # new dict, SAME inner list
>>> deep = copy.deepcopy(orig)                     # fully independent
>>> orig["tags"].append("b")
>>> shallow["tags"]
['a', 'b']
>>> deep["tags"]
['a']
>>> import json
>>> payload = json.dumps({"name": "John", "vip": True, "score": None})
>>> payload
'{"name": "John", "vip": true, "score": null}'
>>> json.loads(payload)["vip"]
True
```

07 Gotchas

```
>>> tmp = {"a": 1, "b": 2}
>>> for k in tmp:
...     del tmp[k]
RuntimeError: dictionary changed size during iteration
>>> for k in list(tmp):                            # loop over a COPY of the keys instead
...     del tmp[k]
>>> tmp
{}
>>> shared = dict.fromkeys(["a", "b"], [])         # every key shares ONE list
>>> shared["a"].append(1)
>>> shared
{'a': [1], 'b': [1]}
>>> safe = {k: [] for k in ["a", "b"]}             # one list per key
>>> safe["a"].append(1)
>>> safe
{'a': [1], 'b': []}
```

- Never add or delete keys while looping over the same dict; loop over `list(d)` instead.
- `==` ignores order: `{"a": 1, "b": 2} == {"b": 2, "a": 1}` is `True`.

08 Sample program: customer profile management

You are building a customer profile manager for a CRM (Customer Relationship Management) system and storing each customer's data in a Python dictionary.

customer_profile.py

```
# Customer profile management (CRM)

# 1. Create the dictionary and print it
customer = {"name": "John Doe", "age": 32, "city": "New York"}
print("1.", customer)

# 2. Add email and phone, then print
customer["email"] = "john.doe@example.com"
customer["phone"] = "+1-555-0100"
print("2.", customer)

# 3. Print the name and city values
print("3. Name:", customer["name"], "| City:", customer["city"])

# 4. Does the key "email" exist?
print("4. 'email' exists:", "email" in customer)

# 5. Delete the age field and print
del customer["age"]
print("5.", customer)

# 6. Keys, values and items
print("6. Keys:  ", customer.keys())
print("   Values:", customer.values())
print("   Items: ", customer.items())

# 7. Remove and print the last inserted key-value pair
last_pair = customer.popitem()
print("7. Removed:", last_pair)

# 8. .get() on a key that does not exist returns None instead of an error
print("8. Membership:", customer.get("membership"))

# 9. Update with a new field and print the final dictionary
customer.update({"address": "221B Baker Street"})
print("9. Final:", customer)
```

Output

```
1. {'name': 'John Doe', 'age': 32, 'city': 'New York'}
2. {'name': 'John Doe', 'age': 32, 'city': 'New York', 'email': 'john.doe@example.com', 'phone':
    '+1-555-0100'}
3. Name: John Doe | City: New York
4. 'email' exists: True
5. {'name': 'John Doe', 'city': 'New York', 'email': 'john.doe@example.com', 'phone':
    '+1-555-0100'}
6. Keys:   dict_keys(['name', 'city', 'email', 'phone'])
   Values: dict_values(['John Doe', 'New York', 'john.doe@example.com', '+1-555-0100'])
   Items:  dict_items([('name', 'John Doe'), ('city', 'New York'), ('email',
    'john.doe@example.com'), ('phone', '+1-555-0100')])
7. Removed: ('phone', '+1-555-0100')
8. Membership: None
9. Final: {'name': 'John Doe', 'city': 'New York', 'email': 'john.doe@example.com', 'address':
    '221B Baker Street'}
```

What each step demonstrates

#	Task	Dictionary feature
1	Create `customer` with name, age and city, then print it	Dictionary literal
2	Add `email` and `phone`, then print the dictionary	`d[key] = value`
3	Print the customer's name and city	`d[key]`
4	Check whether the key `email` exists	`in`
5	Delete the `age` field and print the dictionary	`del d[key]`
6	Print all keys, values and items	`keys()`, `values()`, `items()`
7	Remove and print the last inserted pair	`popitem()`
8	Use `.get()` on the missing key `membership`	`get()`
9	Update with `address` = 221B Baker Street, print the final dictionary	`update()`

NOTE The email and phone values are sample data. Step 7 removes `phone`, because it was the last pair inserted before the final update.

Bonus: friendlier output

customer_profile_bonus.py

```
# Continues the program above

print("Membership:", customer.get("membership", "Not a member"))   # custom default

for field, value in customer.items():
    print(f"{field:>8}: {value}")

print("Fields stored:", len(customer))
```

Output

```
Membership: Not a member
    name: John Doe
    city: New York
   email: john.doe@example.com
 address: 221B Baker Street
Fields stored: 4
```